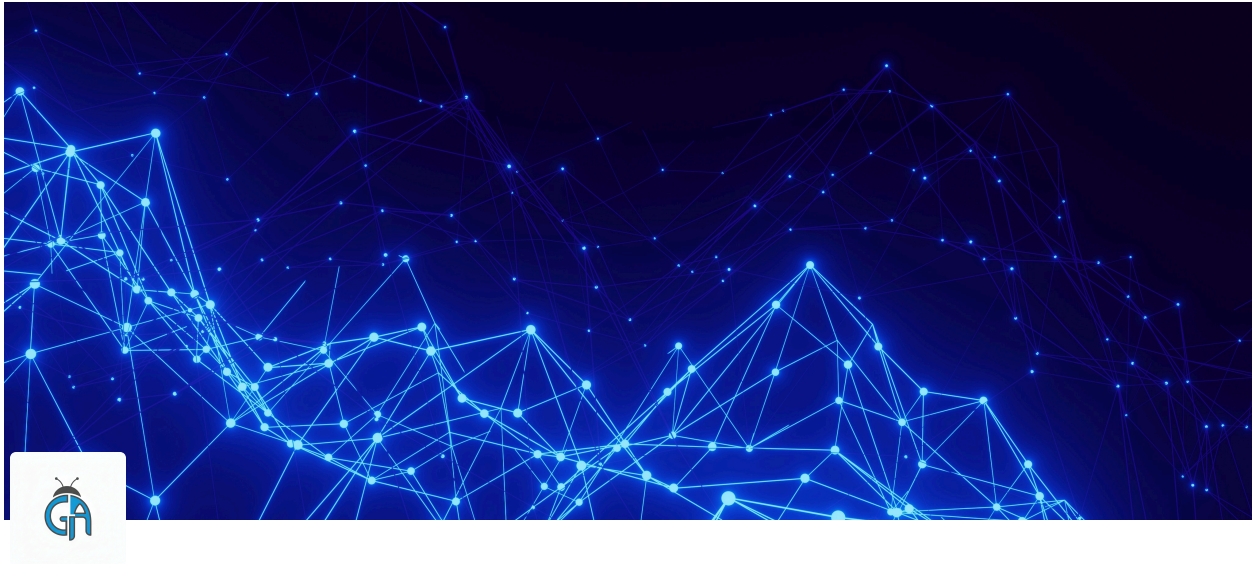




△ The Test Pyramid

Why We Don't Automate Everything

👉 **Author:** Georgia Antoniou

🏷️ **Tags:** QA, Automation, Concepts

🕒 **Reading Time:** 2 min

🚀 **TL;DR**

Tests are not all the same.

Unit Tests are cheap and fast (Do lots of these).

Manual/UI Tests are expensive and slow (Do fewer of these).

The Rule: Catch bugs as low as possible.

🧠 **The Simple Concept**

Imagine a Pyramid divided into three layers. The shape tells us **how many** tests we should have.

1. The Base (Unit Tests):

- - **What:** Testing tiny pieces of code (like a single math calculation).
 - **Speed:** Instant.
 - **Strategy:** We should have **thousands** of these. They are the solid foundation.

2. The Middle(Integration Tests):

- - **What:** Testing if two things talk to each other (e.g., Does the App talk to the Database?).
 - **Speed:** Medium.
 - **Strategy:** We need a good amount of these.

3. The Top(End-To-End/UI Tests):

- - **What:** Opening the browser and clicking buttons like a real user.
 - **Speed:** Slow.
 - **Strategy:** We should have **few** of these. Only test the critical flows here.

The House Analogy

Think of software like building a house.

Unit Tests (The Bricks):

Before you build a wall, you check if the individual bricks are solid.

It's easy to throw away one bad brick.

Integration Tests (The Walls):

You check if the bricks stick together to make a wall.

If the wall falls down, it's harder to fix than a single brick.

UI Tests (The Roof):

You check the finished house.

If you find a crack in the foundation now, you have to tear the whole house down to fix it. That is expensive!

Real World Example: A Calculator

Testing the "Add" Function ($2 + 2 = 4$)

- **Unit Test:** The computer checks the math logic internally.
 - - *Time:* 0.001 seconds.
- **UI Test:** A robot opens the Calculator app, waits for it to load, moves the mouse, clicks "2", clicks "+", clicks "2", clicks "=".
 - - *Time:* 5 seconds.

Conclusion: Both tests check the same thing, but the UI test is **5000x slower**. This is why we push testing to the bottom of the pyramid.

Why This Matters

Many teams try to test *everything* through the UI. This creates the "Ice Cream Cone" problem (heavy on top).

It makes releases slow because testing takes days.

By following the Pyramid, we catch bugs instantly (at the bottom), so the Manual QA (me) can focus on the complex, interesting stuff at the top.